챕터 7. 릴 필름처럼 나눠 틀기. HLS + FFmpeg

이번 챕터가 끝나면

- mp4 한 덩어리를 정적 URL로 내려 주는 방식이 **왜 무너지는지**를 체감합니다(대역폭·시크 지연·모바일 취약)
- FFmpeg로 원본을 **10초 단위 ts 조각 + m3u8 재생목록**으로 자르는 법을 손으로 해 보겠습니다
- Spring에서 업로드 → 비동기 인코딩 → HLS 정적 제공으로 이어지는 파이프라인을 컨트롤러 두 개와 서비스 한 개로 구성합니다
- Postman과 브라우저 DevTools Network 탭에서 `video.m3u8` 다음에 `video0.ts`, `video1.ts`, ...가 순서대로 내려오는 모습을 직접 확인합니다
- HLS·DASH·WebRTC·RTMP·RTSP를 "지연·방향·용도" 세 축으로 한 장짜리 비교표로 정리하고, 챕터 6의 WebSocket과 대비합니다

이번 챕터는 "영화관 영사기가 한 통의 필름을 열두 통으로 나눠 순서대로 돌리는 이야기"입니다

챕터 6 마지막에서 오픈이는 세 방식(Polling · SSE · WebSocket)을 나란히 놓고 "어느 자리에 무엇이 맞는가"를 정리했습니다. 수첩을 덮으려는데 동료가 다음 주 회의를 꺼냈습니다. 강의 VOD를 호스팅해야 하는데, 업로드한 mp4를 그대로 내려 주는 구현이 3분째 버퍼링으로 멈춘다는 것이었습니다. 이 챕터에서는 그 3분을 끊어 보겠습니다. 서버가 영상 한 덩어리를 통째로 밀어 주는 대신, FFmpeg가 10초짜리 필름 조각으로 잘라 두고, 브라우저가 조각을 순서대로 집어 가게 만듭니다. 같은 영상이, 같은 회선에서, 거의 끊기지 않고 돕니다. 끝에서는 HLS가 놓인 자리를 WebRTC·RTMP와 함께 지도 위에 놓고, "언제 무엇을 고를지"를 한 장으로 정리합니다.

준비하기. 실습 시작 전 한 번만 설정

1. 소스 코드 준비

챕터 7은 Spring Boot 하나로 시작합니다. 챕터 6까지 다뤘던 실시간 통신 레포와 별개의 레포입니다. 이번 레포는 업로드 API 하나와 HLS 정적 제공 컨트롤러 두 개가 핵심입니다.

레포	용도	주소
spring-hls	Spring Boot + FFmpeg 자바 래퍼 + HLS 정적 매핑	`github.com/metacoding-11-spring-reference/spring-hls`

터미널에서 클론합니다.

터미널 실습 레포 클론

```
git clone https://github.com/metacoding-11-spring-reference/spring-hls.git
cd spring-hls
```

파일 구조는 이렇습니다.

spring-hls 디렉토리

```
spring-hls/
├── build.gradle                                  # [실습] net.bramp.ffmpeg 의존성
├── src/main/
│   ├── java/com/metacoding/hls/
│   │   ├── HlsApplication.java                   # [실습] @EnableAsync
│   │   ├── controller/HlsController.java         # [실습] /upload · /hls/{quality}/*.m3u8 · *.ts
│   │   └── service/HlsService.java               # [실습] 저장 · 인코딩 · 로딩
│   └── resources/
│       ├── application.properties                # [실습] multipart 2GB
│       └── templates/index.mustache              # [참고] Hls.js 플레이어
└── upload/                                       # 런타임 생성. original/ · hls/720p/ · hls/1080p/
```

`upload/` **폴더는 레포에 커밋되지 않습니다.** 첫 업로드가 들어올 때 `mkdirs()`로 자동 생성되고, 그 아래로 `original/video.mp4`와 `hls/720p/video.m3u8`·`video0.ts` 같은 파일들이 쌓입니다. 실습이 끝난 뒤 용량이 부담되면 이 폴더를 통째로 삭제하면 됩니다.

2. 실습 환경 구축

터미널 실습 환경 확인

```
java --version
./gradlew --version
ffmpeg -version
```

FFmpeg는 호스트에 설치되어 있어야 합니다. 이 프로젝트는 FFmpeg 실행 파일을 자바 코드에서 `ProcessBuilder`로 부르는 구조이기 때문에, 시스템 PATH 위에 `ffmpeg` 명령이 잡혀 있어야 합니다.

- macOS: `brew install ffmpeg`
- Ubuntu: `sudo apt-get install ffmpeg`
- Windows: 공식 빌드(gyan.dev)를 내려받아 PATH 등록

브라우저는 크롬 또는 엣지를 권장합니다. 사파리는 HLS를 네이티브 지원하지만, DevTools Network 탭의 요청 흐름은 크롬에서 더 명확하게 보입니다.

FFmpeg를 도커 컨테이너로 쓰는 선택지

호스트 환경을 오염시키고 싶지 않다면 `jrottenberg/ffmpeg:6.0-ubuntu` 같은 이미지를 `docker run`으로 돌리는 방법도 있습니다. 다만 이 실습은 자바 래퍼(`net.bramp.ffmpeg`)가 로컬 `ffmpeg` 바이너리를 직접 호출하는 구조라, 컨테이너로 쓰려면 Spring 서비스 코드도 Docker Exec 방식으로 바꿔야 합니다. 이 챕터에서는 호스트 설치를 기본값으로 갑니다.

3. 사용할 구성 요소

이번 챕터에 새로 얹히는 재료는 여섯 가지입니다.

재료	역할
`net.bramp.ffmpeg:ffmpeg:0.8.0`	FFmpeg 명령을 자바 코드에서 다루기 쉽게 해 주는 래퍼
`FFmpegBuilder`	입력·출력·추가 옵션(`-hls_time`, `-vf scale`...)을 메서드 체이닝으로 조립
`MultipartFile.transferTo`	업로드된 파일을 실제 디스크로 옮기는 Spring 기본 API

`@Async` + `@EnableAsync`	업로드 응답은 즉시, 인코딩은 백그라운드 스레드로
`application/vnd.apple.mpegurl`	m3u8 재생목록의 공식 MIME 타입
`Hls.js`	크롬·엣지에서 HLS를 재생하기 위한 JavaScript 라이브러리 (CDN)

4. 실습 순서

이번 챕터는 이 순서로 흘러갑니다.

1. `7.1` . 동료의 "3분째 버퍼링이에요"
2. `7.2` . 한 통의 필름을 그대로 내리면 벌어지는 일 (Try/Fail)
3. `7.3` . 영화관 영사기는 릴을 나눠 돌린다 (HLS의 비유)
4. `7.4` . FFmpeg로 한 영상을 두 화질·여러 조각으로 자르기
5. `7.5` . Spring 업로드 API와 비동기 인코딩 트리거
6. `7.6` . m3u8·ts를 정적으로 내려 주는 컨트롤러 두 개
7. `7.7` . Postman과 브라우저로 전체 흐름 검증
8. `7.8` . HLS·DASH·WebRTC·RTMP·RTSP 한 장 비교와 선택 가이드

1에서 문제를 듣습니다. 2에서 한 덩어리 내려 주기를 직접 해 보고 무너지는 지점을 확인합니다. 3에서 HLS의 비유를 잡고, 4~6에서 FFmpeg·업로드·정적 매핑을 한 조각씩 얹습니다. 7에서 Postman으로 API를 두드리고 브라우저로 결과를 확인합니다. 8에서 스트리밍 기술 5종을 지도 위에 놓고 이 챕터가 고른 길을 되짚습니다.

7.1 "3분째 버퍼링이에요"

오픈이는 챕터 6을 닫고 수첩을 서랍에 넣었습니다. 월요일 아침, 평소보다 한 잔 더 진한 커피를 뽑아 자리로 돌아오는데, 동료가 노트북을 들고 쫓아왔습니다.

동료 : "업로드한 영상 재생 누르면 3분째 버퍼링이에요."

오픈이 : "몇 메가짜리요?"

동료 : "800MB요. 1시간짜리 강의요."

동료의 크롬 창에는 까만 비디오 영역 한가운데에 스피너가 돌고 있었습니다. DevTools를 열어 Network 탭을 보니 요청은 딱 하나였습니다. `GET /videos/lecture.mp4`. 800MB짜리 응답이 아직 진행 중이었고, 다운로드 진행률은 18%에서 멈춰 있었습니다.

한 덩어리로 내려받는 중이다.

오픈이는 그 자리에서 자기 노트북에 같은 파일을 열어 봤습니다. 재생 버튼을 눌렀습니다. 비디오 요소의 `currentTime`은 0 근처에서 움직이지 않았습니다. 30초를 기다렸다가 타임라인을 정확히 30분 지점으로 드래그했습니다. 스피너가 다시 10초를 돌았습니다.

동료 : "그리고요, 모바일에서 열면 아예 로딩도 안 돼요."

오픈이 : "공용 LTE였죠?"

동료 : "네."

팀장이 지나가면서 화면을 힐끔 봤습니다.

팀장 : "유튜브가 동영상 한 덩어리를 통으로 내려 줄까요? 아니면 잘라서 줄까요?"

오픈이는 바로 대답이 나오지 않았습니다.

팀장 : "유튜브 영상을 볼 때, 앞부분은 봤는데 뒷부분은 아직 안 내려왔을 수도 있잖아요. 그때 사용자가 뒤로 감으면요."

오픈이 : "바로 거기로 가죠."

팀장 : "그 '바로'가 어떻게 될까요. 서버가 800MB를 다 주고 있었다면요."

팀장은 그 말만 남기고 자기 자리로 돌아갔습니다.

잘라서 줘야 한다. 어떤 조각이든 바로 꺼낼 수 있게.

오픈이는 자리에 앉아 지금 구현을 다시 봤습니다. 서버에는 `@GetMapping("/videos/{name}")` 컨트롤러 하나가 있었고, 그 안에서 `FileSystemResource`로 mp4 파일 하나를 응답으로 돌려주고 있었습니다. 브라우저의 `<video src="/videos/lecture.mp4">`가 그 URL을 걸어 두면, HTTP는 처음부터 끝까지 바이트를 순서대로 흘려 보냈습니다. Range 헤더로 일부만 요청할 수는 있었지만, 플레이어는 그 방식을 영리하게 쓰지 못하고 있었습니다. 네트워크가 조금이라도 불안하면 이어받기가 제대로 안 됐습니다.

한 통의 필름을 통째로 내려 주는 구조다.

7.2 한 통의 필름을 그대로 내리면

현재 구조를 정리하면 이렇습니다. 업로드된 mp4가 서버 디스크 한 곳에 놓이고, 서버는 그 파일을 `GetMapping`으로 정적 노출합니다. 브라우저의 `<video>` 태그가 그 URL을 걸면, 브라우저가 파일 전체를 내려받기 시작합니다. HTTP Range 요청으로 부분 받기가 일부 되긴 하지만, 파일은 논리적으로 **한 덩어리**입니다.

이 구조가 세 지점에서 무너집니다.

첫째, **대역폭**. 800MB 파일을 한 덩어리로 본다는 말은, 이 영상을 보는 사람 한 명당 서버가 800MB를 풀로 밀어낼 준비를 해야 한다는 뜻입니다. 100명이 동시에 보면 서버 업로드 대역폭이 100배 곱해져 요구됩니다. CDN 없이 혼자 버티기 어렵습니다.

둘째, **시크 지연**. 사용자가 타임라인을 30분 지점으로 끌면, 브라우저는 "여기부터 주세요" 신호(Range 헤더)를 서버에 보냅니다. 서버가 해당 바이트 오프셋부터 흘려 보내기 시작하는데, mp4는 메타데이터(`moov` 아톰)가 파일 끝이나 앞에 몰려 있어서 플레이어가 탐색을 다시 시작해야 합니다. 그 사이 10초가 검은 화면입니다.

셋째, **모바일 환경 취약**. LTE·5G는 터널·엘리베이터·지하에서 순간순간 끊깁니다. 한 덩어리 다운로드가 도중에 끊기면 이어받기가 완벽하게 되는 건 서버·클라이언트·네트워크가 전부 완벽할 때뿐입니다.
실제로는 그대로 멈추거나, 처음부터 다시 받습니다.

[CAPTURE NEEDED: assets/CH07/terminal/07_mp4-single-download.png | 크롬 DevTools Network 탭 스크린샷. /videos/lecture.mp4 요청 하나만 Pending 상태로 걸려 있고, Size 칸에 "147 MB / 800 MB"처럼 진행 중인 상태. 오른쪽 Timing 탭에 Waiting 30s, Content Download 진행 중으로 표시. 한 덩어리 다운로드가 멈춰 있는 모습을 보여주는 것이 목적]

그림 7-1. mp4 한 덩어리를 받다가 18%에서 멈춘 다운로드 요청입니다

— 문제 정리 —

1. 한 덩어리 다운로드의 비용
   - 사용자 한 명당 풀 대역폭이 할당됨
   - 서버 업로드 대역폭이 동시 시청자 수에 선형 증가
2. 시크 지연
   - 타임라인 드래그 → Range 요청 → 메타데이터 재탐색 → 10초 검은 화면
3. 끊긴 연결 이어붙이기
   - LTE/5G가 순간 끊기면 처음부터 다시 받는 경우가 흔함

동료가 뒤에서 한마디를 더 얹었습니다.

동료 : "유튜브는 왜 안 이래요?"

유튜브는 잘라서 보내고 있었다.

한 번 더 시도해 볼 수 있는 길이 있었습니다. mp4를 그대로 두더라도 HTTP Range 요청을 서버가 제대로 처리하도록 튜닝하면, 브라우저가 파일 중간을 정확히 집어 갈 수 있습니다. `FileSystemResource`를 `ResourceRegion`으로 감싸고 `Accept-Ranges` 헤더를 붙이면 됩니다. 몇 줄이면 되는 수정이었습니다.

그런데 문제는 남았습니다. 여전히 사용자 한 명당 풀 대역폭이 할당됐고, 네트워크가 끊기면 이어받기가 불안정했습니다. `moov` 아톰의 위치에 따라 시크 속도가 갈렸고(영상을 `faststart` 옵션으로 재인코딩해야 앞쪽으로 옮겨집니다), 모바일에서 공용 와이파이가 자주 끊기는 상황은 여전히 풀리지 않았습니다.

반쯤 해결되고 반쯤 남는다.

오픈이는 Range 튜닝을 접었습니다. 필요한 건 "한 덩어리를 잘 쪼개는 법"이 아니라 "처음부터 잘려 있는 구조"였습니다.

7.3 영화관 영사기는 릴을 나눠 돌린다

오픈이는 수첩에 그림을 하나 그렸습니다. 왼쪽에 커다란 필름 릴 통 하나를 그리고, 화살표로 그 옆에 작은 릴 통 열두 개를 그렸습니다.

옛날 영화관은 한 편의 영화를 **여러 통의 릴**로 나눠 돌렸습니다. 한 통이 한 번에 약 20분짜리였고, 영사기사는 1번 릴이 끝나기 전에 2번 릴 영사기에 필름을 걸어 두었습니다. 1번이 끝나는 순간 2번이 이어받고, 관객은 끊김을 느끼지 못합니다. 중간에 필름이 끊어져도 1번 릴 전체가 다시 시작되지 않았습니다. 끊어진 그 릴만 새 걸로 갈아 끼우면 됐습니다.

HLS(HTTP Live Streaming)는 이 영사실을 웹으로 옮겨 놓은 방식입니다. 서버가 영상을 10초짜리 조각(`.ts` 파일) 여러 개로 미리 잘라 두고, "어떤 순서로 틀어야 하는지"를 적은 재생목록(`.m3u8` 파일)을 한 장 같이 둡니다. 브라우저는 먼저 m3u8을 받아서 목록을 읽은 뒤, 그 안에 적힌 ts를 한 장씩 순서대로 내려받습니다.

[GEMINI PROMPT: 영화관 영사실 장면. 왼쪽 벽에 "한 통의 필름" 레이블이 붙은 커다란 릴 통 하나가 영사기 한 대에 걸려 있다. 화살표(변환 표시)를 지나 오른쪽에는 "HLS" 레이블 아래 작은 릴 통 여섯~여덟 개가 선반에 가지런히 놓여 있고, 그 앞에 "video.m3u8"이라고 적힌 목록 종이 한 장이 펼쳐져 있다. 종이에는 "video0.ts", "video1.ts", "video2.ts"...가 줄지어 적혀 있다. 영사기사(오픈이 캐릭터)가 종이를 들여다보며 1번 릴을 영사기에 거는 모습. 따뜻한 실내등, 흰 바닥, 인디고·오렌지 악센트, 부드러운 일러스트 톤.]

그림 7-2. 한 통의 필름을 여러 개로 나눠 두고, 목록(m3u8) 순서대로 돌립니다

이 구조가 앞 절에서 본 세 지점을 정확히 풀어 줍니다.

첫째, 대역폭. 플레이어가 한 번에 받는 건 한 조각(10초, 대략 2~5MB)입니다. 800MB를 통으로 밀어낼 일이 없고, 중간에 사용자가 창을 닫아도 서버는 다음 조각을 보낼 준비만 멈추면 됩니다.

둘째, 시크 지연. 사용자가 30분 지점을 클릭하면, 플레이어는 m3u8에서 "30분에 해당하는 조각은 `video180.ts`"라는 정보를 계산하고, 그 조각만 바로 요청합니다. 그 앞의 179개 조각은 받을 필요가 없습니다. 거의 즉시 재생이 시작됩니다.

셋째, 끊긴 연결. 한 조각을 받다 실패하면 그 조각만 다시 받으면 됩니다. 앞의 30분은 이미 브라우저 버퍼에 있고, 그 뒤 한 장만 재시도합니다. 플레이어에 내장된 재시도 로직이 이걸 자동으로 처리합니다.

그림 7-3. 한 통 vs 여러 조각. 같은 영상을 두 방식으로 내려받을 때

한 덩어리 (mp4)

요청 1회(또는 Range 분할). 첫 재생까지 파일 프리롤 필요. 시크는 Range 재요청 + 메타 재탐색. 연결

끊기면 처음부터 다시 받는 경우가 흔함. 적응형 화질 없음.

여러 조각 (HLS)

m3u8 1회 + ts N회(10초당 1개). 1~2개 조각만 받으면 재생 시작. 시크는 해당 조각만 바로 요청. 끊겨도 실패한 한 조각만 재시도. m3u8에 여러 화질 등록 시 네트워크에 따라 자동 전환.

같은 영상, 다른 배달 방식이 만드는 차이입니다

한 통을 여러 통으로 나누고, 목록을 한 장 얹는다. 그게 HLS다.

남은 질문은 "어떻게 자르나"였습니다. 답은 FFmpeg 한 줄이었습니다.

7.4 FFmpeg로 한 영상을 두 화질·여러 조각으로

FFmpeg는 영상·음성을 변환해 주는 도구입니다. mp4를 mp3로 바꾸거나, 화질을 낮춰 용량을 줄이거나, 이 챕터처럼 **HLS 포맷으로 잘라 내는** 일을 명령 한 줄로 합니다. 커맨드라인 바이너리 하나지만, 자바에서 쓰기 편하게 해 주는 래퍼 라이브러리가 있습니다. 이 프로젝트는 `net.bramp.ffmpeg:ffmpeg:0.8.0`을 씁니다.

먼저 FFmpeg가 HLS를 만들 때 쓰는 옵션을 익히고, 그 옵션들이 어떻게 자바 코드로 번역되는지 살펴보겠습니다.

7.4.1 HLS를 만드는 FFmpeg 옵션

터미널에서 단독으로 HLS를 만든다면 명령은 이렇습니다.

터미널 FFmpeg로 mp4를 HLS로 자르기. 참고용

```
ffmpeg -i input.mp4 \
  -vf scale=-2:720 \
  -b:v 2500k -maxrate 2500k -bufsize 5000k \
  -hls_time 10 -hls_list_size 0 \
  -f hls output.m3u8
```

핵심은 다섯 줄입니다.

옵션	뜻
`-vf scale=-2:720`	세로를 720px에 맞추고 가로는 비율 유지 (`-2`는 "2의 배수로 자동")
`-b:v 2500k`	비디오 비트레이트 목표 2500kbps
`-maxrate 2500k` / `-bufsize 5000k`	비트레이트 변동 상한과 버퍼 크기
`-hls_time 10`	**10초 길이의 ts 조각 하나씩 만듦**
`-hls_list_size 0`	m3u8에 **모든** 조각을 담음 (VOD)
`-f hls`	출력 포맷을 HLS로 지정

`hls_list_size`를 `0`이 아닌 작은 수(예: 5)로 두면 가장 최근 5개 조각만 목록에 남고 나머지는 주기적으로 제거됩니다. 라이브 스트리밍에서 쓰는 모드입니다. 이번 실습은 VOD(녹화된 영상 다시보기)이므로 `0`으로 두고 **전부** 목록에 박아 둡니다.

이 명령을 자바 코드로 옮기면 이렇게 됩니다.

7.4.2 FFmpegBuilder로 옮기기

`src/main/java/com/metacoding/hls/service/HlsService.java`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 1 src/main/java/com/metacoding/hls/service/HlsService.java. 720p HLS 빌더 구성

```
// TODO: 720p로 자를 FFmpegBuilder를 구성
// 1. 입력 파일 경로를 setInput으로 지정
// 2. 출력 경로(.m3u8)를 addOutput으로 지정
// 3. 비트레이트·스케일·HLS 옵션을 addExtraArgs로 얹기
FFmpegBuilder builder720 = new FFmpegBuilder()
        .setInput(inputPath)
        .addOutput(output720)
        .addExtraArgs("-b:v", "2500k")
        .addExtraArgs("-maxrate", "2500k")
        .addExtraArgs("-bufsize", "5000k")
        .addExtraArgs("-vf", "scale=-2:720")
        .addExtraArgs("-hls_time", "10")
        .addExtraArgs("-hls_list_size", "0")
```

```
        .addExtraArgs("-f", "hls")
        .done();
```

`FFmpegBuilder`는 위에서 본 커맨드라인 옵션을 메서드 체인으로 쌓는 객체입니다. `setInput`이 `-i input.mp4`에, `addOutput`이 마지막의 `output.m3u8`에, `addExtraArgs("-hls_time", "10")`이 `-hls_time 10`에 각각 1:1 대응합니다. `done()`은 이 출력 파트를 마감하고 다음 출력(예: 1080p)을 이어 붙일 수 있게 합니다.

1080p 출력은 `scale`과 비트레이트만 바뀌고 나머지는 동일합니다. 같은 패턴을 복사해서 아래처럼 두 번째 빌더를 이어 작성합니다.

실습 1 이어서 src/main/java/com/metacoding/hls/service/HlsService.java. 1080p 빌더

```
// 1080p는 scale과 비트레이트만 키움
FFmpegBuilder builder1080 = new FFmpegBuilder()
        .setInput(inputPath)
        .addOutput(output1080)
        .addExtraArgs("-b:v", "5000k")
        .addExtraArgs("-maxrate", "5000k")
        .addExtraArgs("-bufsize", "10000k")
        .addExtraArgs("-vf", "scale=-2:1080")
        .addExtraArgs("-hls_time", "10")
        .addExtraArgs("-hls_list_size", "0")
        .addExtraArgs("-f", "hls")
        .done();

// 두 빌더를 FFmpegExecutor에 넘겨 순차 실행
new FFmpegExecutor(ffmpeg, ffprobe).createJob(builder720).run();
new FFmpegExecutor(ffmpeg, ffprobe).createJob(builder1080).run();
```

`FFmpegExecutor`는 빌더가 조립해 둔 명령줄을 실제로 실행하는 실행기입니다. 내부에서 호스트의 `ffmpeg` 바이너리를 `ProcessBuilder`로 띄우고, 프로세스의 stdout·stderr를 읽어 진행률을 추적합니다. 이 실습은 두 해상도를 순차로 돌리지만, `CompletableFuture.allOf`로 병렬 실행하면 CPU 코어가 남는 만큼 변환이 빨라집니다.

7.4.3 생성 결과 파일 구조

이 코드가 한 번 돌고 나면 디스크에는 아래 구조가 만들어집니다.

변환 후 upload/ 디렉토리

```
upload/
├── original/
│   └── video.mp4                    # 업로드한 원본. 이름은 video.mp4로 고정
└── hls/
    ├── 720p/
    │   ├── video.m3u8               # 720p 재생목록
    │   ├── video0.ts                # 0~10초
    │   ├── video1.ts                # 10~20초
    │   ├── video2.ts
    │   └── ...
    └── 1080p/
        ├── video.m3u8
        ├── video0.ts
        └── ...
```

`baseName`이 `video`로 고정되어 있어서 파일명은 항상 같습니다. 실습은 "최신 영상 하나만" 재생하는 구조이고, 새로 업로드하면 `video.mp4`를 덮어쓰고 ts도 모두 덮어씁니다. 여러 영상을 동시에 서비스하려면 업로드 파일명이나 UUID로 `baseName`을 바꾸는 작업이 따로 필요합니다.

10초는 왜 10초인가

HLS의 `-hls_time` 표준 권장값은 6~10초 사이입니다. 짧을수록 시크가 정교해지고 라이브 지연이 줄지만, 대신 m3u8 목록이 길어지고 조각 수가 많아져 요청 수가 폭증합니다. 길수록 요청 수는 줄지만 시크 반응이 둔해지고 첫 재생까지 더 긴 버퍼가 필요합니다. 애플 HLS 가이드는 6초를, 일반 VOD는 10초를 많이 씁니다. 이 실습은 10초입니다.

7.5 Spring 업로드 API와 비동기 인코딩 트리거

FFmpeg가 어떻게 자르는지는 앞에서 정리했습니다. 이제 Spring 쪽에 두 개의 끝을 만듭니다. 하나는 **업로드를 받아서 저장하고, FFmpeg 인코딩을 시작시키는** 컨트롤러. 다른 하나는 **만들어진 m3u8·ts를 내려 주는** 컨트롤러. 이 절은 첫 번째입니다.

7.5.1 업로드 용량 설정

`src/main/resources/application.properties`를 열고 TODO의 `pass`를 지우고 아래 두 줄을 작성합니다.

실습 2 src/main/resources/application.properties. 2GB 업로드 허용

```
# 파일 하나의 최대 크기
spring.servlet.multipart.max-file-size=2GB
# 요청 전체 크기 (파일 + 기타 필드)
spring.servlet.multipart.max-request-size=2GB
```

Spring Boot 기본값은 파일 1MB·요청 10MB입니다. 강의 영상 800MB가 기본값에 걸려 `MaxUploadSizeExceededException`으로 튕겨 나가는 일을 막기 위해 2GB로 상향합니다. 실서비스에서는 여기서 한 단계 더 나아가, 챕터 5에서 배운 **Presigned URL로 클라이언트가 S3에 직업로드**하는 구조를 쓰는 게 맞습니다. 이 실습은 로컬 디스크 기반으로 흐름 전체를 체감하는 목적이라 Multipart 업로드를 그대로 씁니다.

7.5.2 비동기 활성화

`src/main/java/com/metacoding/hls/HlsApplication.java`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 3 src/main/java/com/metacoding/hls/HlsApplication.java. @EnableAsync 추가

```
// TODO: @Async 메서드가 별도 스레드에서 돌도록 활성화
// 1. 클래스 레벨에 @EnableAsync 어노테이션 추가
@SpringBootApplication
@EnableAsync
public class HlsApplication {
    public static void main(String[] args) {
        SpringApplication.run(HlsApplication.class, args);
    }
}
```

`@EnableAsync`가 없으면 `@Async` 어노테이션이 달린 메서드도 **같은 스레드에서 동기 실행**됩니다. 자바의 Spring 프록시 시스템이 `@Async` 메서드를 별도 스레드에 던지는 건 이 어노테이션 하나로 활성화됩니다. 한 번만 켜 두면 됩니다.

`@Async` 메서드가 쓰는 스레드 풀은 Spring이 기본 제공합니다. 별도 설정 없이 쓰면 `SimpleAsyncTaskExecutor`가 요청당 새 스레드를 띄우는데, 운영 환경에서는 스레드 수가 폭증할 수 있어 `ThreadPoolTaskExecutor`로 바꿔 **풀 크기**를 제한하는 것이 일반적입니다. 이 실습은 기본 풀로도 충분합니다.

7.5.3 업로드 컨트롤러

`src/main/java/com/metacoding/hls/controller/HlsController.java`를 열고 TODO의 `pass`를 지우고 아래 코드를 작성합니다.

실습 4 src/main/java/com/metacoding/hls/controller/HlsController.java. 업로드 엔드포인트

```
// TODO: 업로드 + 인코딩 트리거 엔드포인트 구현
// 1. @RequestParam("file")로 MultipartFile 수신
// 2. 원본 저장 후 savedName 반환
// 3. 저장된 파일명을 바탕으로 비동기 인코딩 호출
@PostMapping("/upload")
public ResponseEntity<String> uploadVideo(
        @RequestParam("file") MultipartFile file) throws IOException {
    String savedName = hlsService.saveOriginalVideo(file);
    hlsService.convertToHlsAsync(savedName);
    return ResponseEntity.ok("HLS 변환 시작: " + file.getOriginalFilename()
            + " → " + savedName);
}
```

이 짧은 메서드에서 일어나는 일은 셋입니다. Postman이 `form-data`의 `file` 키로 업로드한 바이너리를 `MultipartFile`이 받아 들고, `saveOriginalVideo`가 그걸 `upload/original/video.mp4`로 떨어뜨립니다. 그 다음 `convertToHlsAsync`가 호출되자마자 메서드는 바로 `ResponseEntity.ok(...)`를 반환합니다. **인코딩은 뒤에서 따로 돕니다.** 업로드 요청은 몇 밀리초 안에 응답받고, 사용자는 다른 걸 하러 갈 수 있습니다.

동기 방식으로 짰다면 `uploadVideo`가 FFmpeg가 720p·1080p 두 변환을 모두 마칠 때까지 블록됩니다. 800MB 영상이면 30초~1분이 걸리는데, 그동안 요청이 매달려 있고 HTTP 타임아웃(기본 30초) 위험에 걸립니다. `@Async`는 이 문제를 공간(스레드)으로 해결합니다.

7.5.4 원본 저장과 비동기 변환 서비스

다시 `HlsService.java`를 열고 TODO의 `pass`를 지우고 아래 두 메서드를 작성합니다.

실습 5 src/main/java/com/metacoding/hls/service/HlsService.java. 원본 저장 + 비동기 변환

```
// TODO: 업로드된 파일을 upload/original/video.mp4로 저장
// 1. 저장 디렉토리 생성
// 2. 파일명을 video.mp4로 고정
// 3. transferTo로 디스크에 쓰기
public String saveOriginalVideo(MultipartFile file) throws IOException {
    new File(ORIGINAL_DIR).mkdirs();
    String fileName = "video.mp4";
    File saveFile = new File(ORIGINAL_DIR + fileName);
    file.transferTo(saveFile);
    return fileName;
}

// TODO: FFmpeg 변환을 백그라운드 스레드에서 실행
// 1. @Async로 표시하여 별도 스레드로 분기
// 2. convertToHls를 호출하고 CompletableFuture로 감싸기
@Async
public CompletableFuture<Void> convertToHlsAsync(String fileName) {
    try {
        convertToHls(fileName);
        return CompletableFuture.completedFuture(null);
    } catch (Exception e) {
        log.error("HLS 비동기 변환 실패: {}", fileName, e);
        return CompletableFuture.failedFuture(e);
    }
}
```

`saveOriginalVideo`는 디스크 I/O입니다. `transferTo`는 Spring이 이미 임시 파일로 받아 둔 multipart 파트를 지정한 경로로 옮깁니다. 파일명을 `video.mp4`로 고정했기 때문에 같은 버킷·같은 이름으로 매번 덮어쓰는 구조입니다.

`convertToHlsAsync`의 `@Async` 한 줄이 이 메서드를 Spring의 `TaskExecutor`에 올려 별도 스레드에서 돌게 만듭니다. `convertToHls`(실습 1의 FFmpegBuilder를 실행하는 메서드)는 이 스레드에서 720p·1080p 두 개를 순차로 뽑고 끝납니다. 실패해도 호출자는 이미 응답을 받아 돌아간 뒤이므로, 에러는 로그와 `CompletableFuture.failedFuture`로만 남습니다.

그림 7-4. 업로드 요청과 인코딩 작업이 분리되는 순서

① Client → /upload form-data로 `file` 파트 전송 (800MB 영상)

② Controller	원본을 `upload/original/video.mp4`로 저장
③ 비동기 분기	`@Async convertToHlsAsync`가 별도 스레드로 출발
④ Controller → Client	200 OK 즉시 반환 ("HLS 변환 시작")
⑤ Worker → FFmpeg	720p · 1080p 두 번 인코딩, 10초 단위 ts 조각 생성
⑥ FFmpeg → 파일시스템	`hls/720p/video.m3u8` + `video0.ts ~ videoN.ts` 떨어짐

응답은 ④에서 끝나고, ⑤·⑥은 백그라운드에서 이어집니다

> **상태 알림은 이 챕터 범위가 아닙니다**
> 업로드 응답 시점에는 인코딩이 아직 진행 중이므로, 사용자가 재생을 누르는 타이밍과 변환 완료 시점 사이가 비동기입니다. 실제 서비스라면 챕터 6에서 만든 SSE로 "10%··50%··완료" 진행률을 밀어 주는 게 정석입니다. 이번 챕터는 핵심(HLS 조각화·정적 제공)에 집중하기 위해 상태 API 대신 **재생 URL을 직접 요청해서 200/404로 간접 확인**하는 구조를 씁니다.

7.6 m3u8·ts를 정적으로 내려 주는 컨트롤러 두 개

FFmpeg가 디스크에 m3u8과 ts를 떨어뜨려 놓았다면, Spring이 해야 할 일은 단순합니다. **브라우저가 요청한 파일을 그대로 내려 주기**. 다만 MIME 타입이 두 가지로 갈립니다. m3u8은 애플이 지정한 전용 타입을, ts는 일반 바이너리 타입을 씁니다.

7.6.1 m3u8 재생목록 엔드포인트

다시 `HlsController.java`를 열고 TODO의 `pass`를 지우고 아래 메서드를 작성합니다.

실습 6 src/main/java/com/metacoding/hls/controller/HlsController.java. m3u8 내려 주기

```
// TODO: /hls/{quality}/{fileName}.m3u8 요청을 받아 파일로 응답
// 1. PathVariable로 quality(720p/1080p)와 fileName(video) 분리
// 2. 서비스에서 Resource 로드
// 3. Content-Type을 application/vnd.apple.mpegurl로 설정
```

```
@GetMapping("/hls/{quality}/{fileName}.m3u8")
public ResponseEntity<Resource> getHlsPlaylist(
        @PathVariable String quality,
        @PathVariable String fileName) throws IOException {
    Resource resource = hlsService.loadHlsFile(quality, fileName + ".m3u8");
    HttpHeaders headers = new HttpHeaders();
    headers.setContentType(MediaType.parseMediaType("application/vnd.apple.mpegurl"));
    return new ResponseEntity<>(resource, headers, HttpStatus.OK);
}
```

핵심은 `application/vnd.apple.mpegurl`이라는 MIME 타입 한 줄입니다. 브라우저의 `Hls.js`나 사파리 네이티브 플레이어가 이 타입을 보고 "아, m3u8이니까 목록을 파싱해서 ts를 이어 받자"라고 판단합니다. 타입이 잘못되면 플레이어는 이 응답을 일반 텍스트로 오해하고 재생을 시도하지 않습니다.

`PathVariable`이 두 개 쪼개진 이유는 URL 구조를 `/hls/720p/video.m3u8` 형태로 고정했기 때문입니다. `quality`는 서비스가 어느 폴더(`CONVERT_DIR_720` 또는 `CONVERT_DIR_1080`)에서 파일을 읽을지 결정하고, `fileName`은 확장자 앞의 이름(`video`)입니다. 둘을 조합해서 `upload/hls/720p/video.m3u8` 실제 경로를 만듭니다.

7.6.2 ts 세그먼트 엔드포인트

같은 파일에 두 번째 메서드를 이어 작성합니다.

실습 7 src/main/java/com/metacoding/hls/controller/HlsController.java. ts 조각 내려 주기

```
// TODO: /hls/{quality}/{tsSegment}.ts 요청을 받아 바이너리로 응답
// 1. PathVariable로 quality와 tsSegment(video0, video1 ...) 분리
// 2. 서비스에서 Resource 로드
// 3. Content-Type을 application/octet-stream으로 설정
@GetMapping("/hls/{quality}/{tsSegment}.ts")
public ResponseEntity<Resource> getHlsTs(
        @PathVariable String quality,
        @PathVariable String tsSegment) throws IOException {
    Resource resource = hlsService.loadHlsFile(quality, tsSegment + ".ts");
    HttpHeaders headers = new HttpHeaders();
    headers.setContentType(MediaType.APPLICATION_OCTET_STREAM);
    return new ResponseEntity<>(resource, headers, HttpStatus.OK);
}
```

`.ts`는 실제로는 MPEG-2 Transport Stream이라는 포맷의 바이너리 파일입니다. 브라우저가 단독으로 이 타입을 해석하진 못하지만, `Hls.js`가 받아서 `<video>`의 내부 `MediaSource`에 넣어 재생을 이어 갑니다. 그래서 일반 바이너리 타입(`application/octet-stream`)으로 내려도 충분합니다. 이 URL은 사람이 직접 열 일이 거의 없습니다. 플레이어만 씁니다.

7.6.3 실제 파일 로딩

`HlsService.java`의 마지막 메서드입니다.

실습 8 src/main/java/com/metacoding/hls/service/HlsService.java. 실제 파일 경로 매핑

```
// TODO: quality에 따라 디스크에서 파일을 찾아 Resource로 반환
// 1. quality로 디렉토리 분기
// 2. File 객체를 만들고 존재 여부 확인
// 3. FileSystemResource로 감싸 반환
public Resource loadHlsFile(String quality, String fileName) throws IOException {
    String path;
    if (quality.equals("720p")) path = CONVERT_DIR_720;
    else if (quality.equals("1080p")) path = CONVERT_DIR_1080;
    else throw new RuntimeException("지원하지 않는 해상도");
    File file = new File(path, fileName);
    if (!file.exists()) throw new IOException("파일 없음: " + file.getAbsolutePath());
    return new FileSystemResource(file.getCanonicalPath());
}
```

컨트롤러가 얇은 대신 서비스에 책임이 몰려 있는 구조입니다. `quality`가 유효한지, 파일이 실제로 존재하는지를 여기서 검증합니다. 인코딩이 아직 진행 중이라 파일이 없으면 `IOException`으로 던지고, Spring의 기본 예외 매핑이 클라이언트에 5xx 응답을 돌려줍니다. 변환 완료 여부를 간접 확인하는 용도로 "m3u8 URL을 GET 해 봐서 200이면 완료"라는 규칙이 여기서 나옵니다.

7.7 Postman과 브라우저로 전체 흐름 검증

서버를 올리고, Postman으로 업로드하고, 브라우저로 재생합니다. 세 단계입니다.

7.7.1 서버 실행

터미널 spring-hls 실행

```
./gradlew bootRun
```

로그에 `Started HlsApplication`이 찍히고 `8080` 포트에서 수신 준비가 되면 다음 단계로 갑니다. 이 시점에 `upload/` 폴더는 아직 없습니다. 첫 업로드가 들어와야 생깁니다.

7.7.2 Postman으로 업로드 요청

Postman을 열고 `POST http://localhost:8080/upload` 요청을 만듭니다. Body 탭에서 `form-data`를 선택하고 아래 한 줄을 추가합니다.

Key	Type	Value
`file`	**File**	업로드할 mp4 파일 선택 (10~100MB 추천)

Key는 반드시 `file`이어야 합니다. 컨트롤러의 `@RequestParam("file")`과 이름이 맞아야 바인딩됩니다. Type은 드롭다운에서 Text가 아니라 **File**을 고릅니다. 드롭다운이 안 보이면 Key 셀에 마우스를 올리면 나타나는 회색 `Text` 버튼을 클릭해서 전환합니다.

첫 실습 파일은 **10~100MB 사이의 짧은 영상**을 권장합니다. 인코딩 시간이 영상 길이·해상도에 비례하기 때문에 긴 영상으로 테스트하면 첫 사이클이 길어집니다.

[CAPTURE NEEDED: assets/CH07/terminal/07_postman-upload.png | Postman 창. POST http://localhost:8080/upload 요청. Body 탭의 form-data에 file 키가 File 타입으로 설정되어 있고 sample.mp4가 선택된 상태. Send 버튼 클릭 후 응답 영역에 "HLS 변환 시작: sample.mp4 → video.mp4" 200 OK가 나온 화면]

그림 7-5. Postman으로 업로드 요청을 보내고 즉시 200 응답을 받는 모습입니다

응답이 **즉시** 돌아옵니다. 몇 백 밀리초 안에 `HLS 변환 시작: ...` 메시지가 뜨고, 그 사이 IntelliJ 콘솔이나 `tail -f` 중인 로그에는 FFmpeg가 방금 막 돌기 시작했다는 라인이 찍히고 있습니다. 여기서 Postman은 돌려줬지만 **변환은 아직 끝나지 않았습니다.**

7.7.3 변환 상태 간접 조회

별도 상태 API를 두지 않았으므로, 재생 URL 자체가 상태 체크입니다.

터미널 변환 완료 간접 확인

```
curl -I http://localhost:8080/hls/720p/video.m3u8
```

응답 코드를 보고 판단합니다.

- `200 OK`: 720p 변환이 끝났고 플레이어가 재생을 시작할 수 있음
- `500` 또는 `404`: 변환이 아직 진행 중이거나 실패

변환 시간은 영상 길이의 대략 30~50% 정도입니다. 30분짜리 영상이면 10분 안팎이 걸립니다. 이 구간은 SSE로 "30% 진행 중"을 밀어 주면 훨씬 매끄럽지만, 이 실습은 폴링 방식으로 직접 몇 번 다시 눌러 확인합니다.

[CAPTURE NEEDED: assets/CH07/terminal/07_curl-check.png | 터미널 두 번의 curl -I 실행. 첫 번째는 HTTP/1.1 500 Internal Server Error (변환 중). 두 번째는 1분 뒤 HTTP/1.1 200 OK에 Content-Type: application/vnd.apple.mpegurl이 찍혀 있음]

그림 7-6. 처음엔 500, 1분 뒤엔 200. 변환이 끝났다는 뜻입니다

7.7.4 브라우저 재생과 Network 탭 관찰

크롬을 열고 `http://localhost:8080` 에 접속합니다. `index.mustache`의 플레이어가 Hls.js를 로드한 뒤 자동으로 `/hls/720p/video.m3u8`을 요청합니다. 재생이 시작되면 F12를 눌러 DevTools Network 탭을 켭니다.

[CAPTURE NEEDED: assets/CH07/terminal/07_devtools-network.png | 크롬 DevTools Network 탭. 요청 리스트에 video.m3u8이 제일 위에 Content-Type application/vnd.apple.mpegurl로 200, 그 아래 video0.ts, video1.ts, video2.ts, video3.ts가 순차로 200 OK로 내려오는 모습. 각 ts 파일 Size 칸에 2~5MB. Protocol이 h2 또는 http/1.1]

그림 7-7. m3u8 한 번, ts는 순차적으로. HLS가 돌아가고 있다는 증거입니다

눈여겨볼 지점은 둘입니다. 첫째, **요청이 여러 개**입니다. 챕터 초반의 한 덩어리 요청 하나가 아니라, `video.m3u8` 하나 + `video0.ts`, `video1.ts`, … 여러 개가 순차로 내려옵니다. 둘째, **타임라인을 드래그해 보세요.** 앞쪽을 건너뛰고 5분 지점으로 이동하면, `video30.ts` 같은 중간 번호의 조각이 즉시 요청되고 재생이 1초 이내에 시작됩니다. 챕터 초반 mp4 한 덩어리에서 30초씩 걸리던 그 검은 화면이 사라졌습니다.

`http://localhost:8080/hls/1080p/video.m3u8` URL로 바꿔 들어가면 1080p 버전도 같은 방식으로 동작합니다. 두 화질 중 어떤 걸 쓸지는 플레이어 설정이나 사용자 선택에 따라 달라집니다. HLS 표준은 m3u8 한 장에 **여러 화질을 목록으로 묶어 두는 마스터 플레이리스트** 개념도 제공합니다. 네트워크 상태에 따라 플레이어가 자동으로 720p↔1080p를 전환하는 **적응형 비트레이트(ABR)**가 이 구조 위에서 돕니다. 이번 실습은 화질별 m3u8을 따로 제공하는 단순 구조이지만, 상용 서비스는 거의 다 마스터 플레이리스트를 씁니다.

HLS 실습에서 생각해 볼 것들

- **조각 길이(hls_time)**: 10초를 6초로 줄이면 시크 반응이 빨라지지만 요청 수가 늘어납니다. 라이브에서는 2~4초까지 내려갑니다
- **hls_list_size**: VOD는 `0`(전부), 라이브는 `3~5`(최근 몇 개만). 어떤 스트리밍을 하느냐가 이 숫자 하나로 갈립니다
- **MIME 타입**: `application/vnd.apple.mpegurl` 한 줄을 빼먹으면 플레이어가 m3u8을 텍스트로 오인하고 재생을 포기합니다
- **CORS**: 프런트 도메인과 서버 도메인이 다르면 m3u8·ts 둘 다에 `Access-Control-Allow-Origin`이 필요합니다. 이 실습은 같은 호스트라 생략
- **CDN**: 실서비스는 m3u8·ts를 CloudFront 같은 CDN에 얹어 엣지에서 바로 내립니다. Spring은 변환만 하고 배포는 CDN이 담당

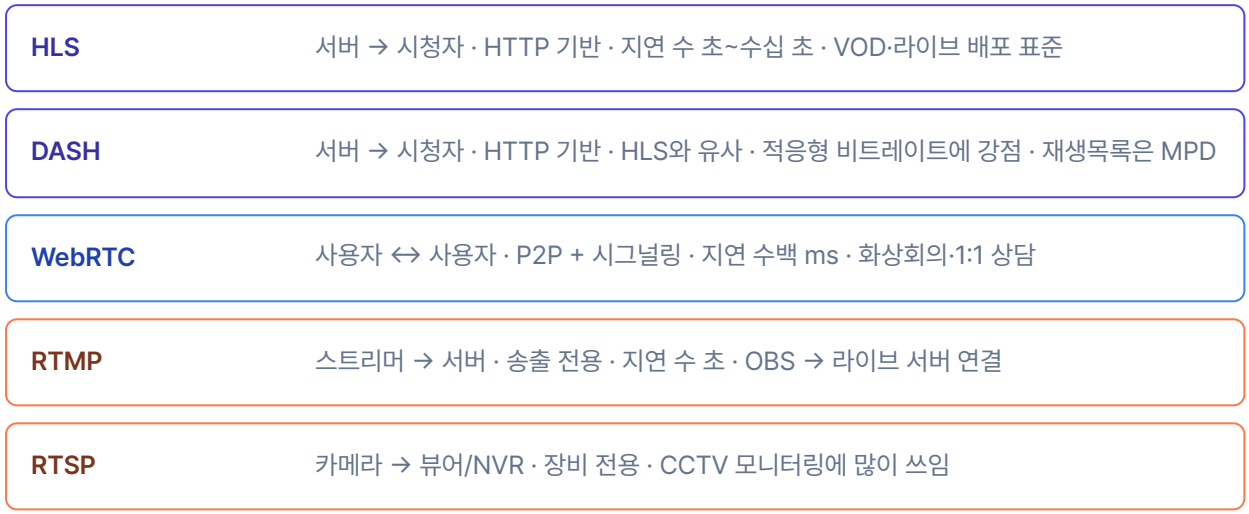
7.8 HLS가 놓인 자리. 스트리밍 기술 한 장 지도

세 개의 터미널 탭과 브라우저 DevTools를 앞에 두고, 오픈이는 수첩을 다시 폈습니다.

한 덩어리에서 여러 조각으로. 요청이 많아진 대신 끊김이 줄었다.

이번 챕터가 고른 건 HLS였습니다. 하지만 HLS가 스트리밍의 전부는 아닙니다. 같은 "영상을 전달한다"는 목표라도, 지연 요건과 통신 방향에 따라 쓰는 기술이 달라집니다. 한 장 지도를 그립니다.

그림 7-8. 스트리밍 기술 5종 한 장 비교



이 표에서 먼저 보이는 축은 **지연**입니다.

- 수 초~수십 초 허용: **HLS**, **DASH** (VOD·라이브 배포)
- 수 초 (송출 쪽): **RTMP** (스튜디오에서 서버로 올릴 때)
- 수백 ms 이하: **WebRTC** (대화·상담·실시간 게임)

두 번째 축은 **방향**입니다. HLS·DASH는 "서버 → 다수의 시청자" 단방향 배포. RTMP는 "한 명의 스트리머 → 서버" 단방향 송출. WebRTC는 "사용자 ↔ 사용자" 양방향. RTSP는 "장비 → 뷰어" 특수 용도.

챕터 6의 WebSocket이 이 지도 안에서 어디에 놓일까요. WebSocket은 **영상 전용이 아닙니다.** 양방향 프레임 채널이고, 그 위에 채팅·협업·신호만 주로 올립니다. 영상·음성 자체를 실시간으로 주고받아야 하는 자리는 WebRTC가 맡습니다. WebRTC는 **시그널링**(누가 누구와 연결할지 약속)을 WebSocket이나 HTTP로 한 뒤, 실제 미디어는 UDP 기반 자체 프로토콜로 주고받습니다. 그래서 "WebSocket과 WebRTC가 헷갈린다"는 질문의 답은 **층이 다르다**입니다. WebSocket은 범용 양방향 채널, WebRTC는 영상·음성 전용 초저지연 양방향.

같은 "업로드한 강의 재생" 문제를 WebRTC로 풀 수 있을까요. 기술적으로는 가능하지만 과한 선택입니다. 시청자가 방송자와 실시간 대화할 필요가 없고, 수백 밀리초의 지연이 문제되지도 않습니다. 많은 사람에게 안정적으로 배포하는 데는 HLS가 압도적으로 싸고 단순합니다.

반대로 "줌 같은 1:1 상담"을 HLS로 풀면요. 10초짜리 ts 조각을 만들어 기다렸다가 내려 주는 동안, 상대방은 이미 10초 전 장면을 보고 있습니다. 대화가 안 됩니다. 이 자리는 WebRTC입니다.

이것만은 기억하자

- **HLS는 "잘라서 목록으로 주는" 단방향 배포**. mp4 한 덩어리 대신 m3u8 재생목록 + 10초짜리 ts 조각 여러 개. 요청 수는 늘지만 대역폭·시크·끊김 복구가 전부 풀립니다
- **FFmpeg가 잘라 내는 핵심 옵션 5개**. `-vf scale`(화질), `-b:v`·`-maxrate`·`-bufsize`(비트레이트), `-hls_time`(조각 길이), `-hls_list_size 0`(VOD는 전부 목록), `-f hls`(포맷 지정)
- **업로드와 인코딩은 분리**. `@EnableAsync` + `@Async`로 FFmpeg 변환을 백그라운드 스레드로 보내고, 업로드 응답은 즉시. 변환 완료는 재생 URL GET의 200/500으로 간접 확인
- **MIME 타입 두 개**. m3u8은 `application/vnd.apple.mpegurl`, ts는 `application/octet-stream`. m3u8 타입을 빼먹으면 플레이어가 재생을 시작하지 않습니다
- **스트리밍 기술 지도**. VOD·라이브 배포는 HLS/DASH, 송출은 RTMP, 실시간 대화는 WebRTC, 장비 모니터링은 RTSP. 이 지도 안에서 이번 챕터는 "서버 → 다수 시청자" 자리에 HLS를 놨습니다

동료가 다시 노트북을 들고 왔습니다.

동료 : "오픈이 님, 버퍼링 없이 재생되네요. 근데요, 이번엔 다른 문제예요."

오픈이 : "말해 봐요."

동료 : "사용자가 업로드한 영상 제목으로 검색을 하려는데요, 지금 DB에 `LIKE '%강의%'`로 걸어 놨더니 영상 10만 개 넘어가니까 조회가 5초씩 걸려요."

오픈이 : "`LIKE`로요."

동료 : "네. 그리고 '강의', '강 의', 'Lecture' 이렇게 섞이면 아예 안 잡혀요. 검색창에 '자바 기초' 쳤는데 '기초 자바'는 안 나온다고 항의가 들어와요."

팀장이 뒤에서 마우스를 한 번 클릭하고 자기 일로 돌아갔습니다.

팀장 : "DB로 검색을 하면 그래요. 검색은 검색 엔진한테 맡겨요."

한 테이블을 풀스캔하던 LIKE 쿼리를, 누군가는 밀리초 안에 끝낸다.

오픈이는 수첩을 넘겼습니다. 업로드·재생·스트리밍까지 길이 이어졌고, 이제 그 영상 위에 앉을 **검색창**이 남아 있었습니다. 다음 챕터에서 Elasticsearch를 얹습니다.

용어 정리

이야기 속 표현	진짜 용어	정식 정의
한 통의 필름	mp4 (MPEG-4 Part 14)	비디오·오디오·자막 등을 하나의 컨테이너로 담는 파일 포맷. 메타데이터(`moov` 아톰)와 데이터(`mdat`)를 한 파일 안에 묶어 둠
여러 통으로 나눈 필름	HLS (HTTP Live Streaming)	애플이 설계한 스트리밍 규격. 영상을 짧은 조각(`.ts`)으로 잘라 HTTP로 배포하고, 순서와 길이를 재생목록(`.m3u8`)에 적어 제공
필름 조각	TS 세그먼트 (MPEG-2 Transport Stream)	HLS에서 영상·음성을 담는 조각 파일. 확장자 `.ts`. 네트워크 전송을 가정한 188바이트 패킷 단위 컨테이너
필름 순서 목록	M3U8 재생목록 (Media Playlist)	HLS 재생목록 포맷. 텍스트 파일로, 조각 파일명과 각 조각의 길이(`#EXTINF`)를 순서대로 기록. MIME 타입은 `application/vnd.apple.mpegurl`
변환기	FFmpeg	영상·음성 변환·스트리밍 도구. 커맨드라인 실행 파일. 인코딩·디코딩·포맷 변환·HLS 조각화 등을 지원

자바용 FFmpeg 리모컨	`net.bramp.ffmpeg:ffmpeg`	FFmpeg를 자바 코드에서 `FFmpegBuilder`·`FFmpegExecutor`로 다루는 래퍼 라이브러리
조각 길이	`-hls_time N`	FFmpeg의 HLS 옵션. 한 조각(`.ts`)의 목표 길이를 N초로 지정. VOD는 6~10, 라이브는 2~4가 흔함
재생목록에 담는 조각 수	`-hls_list_size N`	m3u8에 유지할 조각 개수. `0`은 전부 담기(VOD), `3~5`는 최근 몇 개만(라이브 슬라이딩 윈도우)
화질 조정	`-vf scale=-2:H`	FFmpeg 비디오 필터. 세로를 H 픽셀에 맞추고 가로는 비율 유지(`-2`는 2의 배수로 자동). 예: `scale=-2:720`은 720p
백그라운드 실행	`@Async` + `@EnableAsync`	Spring에서 메서드를 별도 스레드에서 실행시키는 어노테이션 쌍. `@EnableAsync`가 기능을 켜고, `@Async`가 개별 메서드에 적용
한 방향 파일 다운로드	`MultipartFile.transferTo(File)`	Spring Multipart 파일을 지정 경로의 파일로 저장하는 편의 메서드. 내부적으로는 임시 파일을 이동
m3u8 전용 타입	`application/vnd.apple.mpegurl`	M3U8 재생목록의 공식 MIME 타입. 플레이어가 이 타입을 보고 "HLS"로 인식
적응형 재생	ABR (Adaptive Bitrate)	마스터 플레이리스트에 여러 화질을 등록해 두고, 네트워크 상태에 따라 플레이어가 화질을 자동 전환하는 방식
송출 프로토콜	RTMP (Real-Time Messaging Protocol)	스트리머가 라이브 영상을 서버로 올릴 때 쓰는 입력 프로토콜. 어도비가 설계. 송출 도구로 OBS가 대표적

장비 스트리밍	RTSP (Real Time Streaming Protocol)	카메라·NVR 같은 장비가 실시간 영상을 전달할 때 쓰는 프로토콜. CCTV·내부망 환경에서 주로 사용
초저지연 양방향	WebRTC (Web Real-Time Communication)	브라우저 간 영상·음성·데이터 P2P 통신 규격. 수백 ms 이하 지연. 시그널링은 별도 채널(주로 WebSocket) 필요
DASH	MPEG-DASH (Dynamic Adaptive Streaming over HTTP)	MPEG이 표준화한 HTTP 기반 적응형 스트리밍. HLS와 목적은 같고, 재생목록 포맷이 MPD(XML)로 다름

이것만은 기억하자

- **HLS는 "잘라서 목록으로" 내려 주는 방식이다.** mp4 한 덩어리를 `video.m3u8` + `video0.ts`, `video1.ts`, ...로 바꾸는 것이 전부입니다. 나머지 대역폭·시크·재시도 이득은 이 구조에서 자연스럽게 따라옵니다
- **FFmpeg가 자르고 Spring이 내려 준다.** 인코딩은 FFmpeg 한 명령, 제공은 컨트롤러 두 개(`*.m3u8` · `*.ts`). 코드는 짧고 역할은 분명합니다. `@Async`로 인코딩을 분리해야 업로드 응답이 빨라집니다
- **MIME 타입은 플레이어의 언어다.** `application/vnd.apple.mpegurl` 한 줄이 m3u8을 "재생목록"으로, 타입이 누락되면 "텍스트"로 오인됩니다. 변환이 끝나도 재생이 안 되면 가장 먼저 이 줄을 보세요
- **스트리밍 기술은 지연·방향·용도로 고른다.** VOD·라이브 배포는 HLS/DASH, 송출은 RTMP, 실시간 대화는 WebRTC, 장비 모니터링은 RTSP. 챕터 6의 WebSocket과 WebRTC는 층이 다른 도구이므로 섞어 쓰지 마세요
- **다음 문제는 검색이다.** 영상을 올리고 재생하는 파이프라인까지 왔으니, 영상에 딸린 제목·태그·설명을 빠르게 찾는 것이 이어집니다. DB의 `LIKE`로는 10만 건 앞에서 무너집니다. 챕터 8에서 Elasticsearch를 얹습니다